

Predator-prey coevolution simulation

Przemysław Popowski¹

¹AGH Krakow
popowskip@student.agh.edu.pl

2026

Abstract

Understanding predator-prey dynamics is essential for ecological research, but traditional mathematical models often fail to capture the complexities of interactions, individual behaviors, and random chance. This paper presents an agent-based model developed in Python to simulate a virtual ecosystem with grass, prey, and predators. By assigning energy costs to actions such as movement and reproduction, the simulation provides a realistic, individual representation of survival and population cycles. Furthermore, each agent carries an evolvable genotype; offspring inherit their parent's genes with random Gaussian mutations, driving a co-evolutionary arms race between predators and prey without any pre-programmed target behavior. The project features a fully functional visual environment coupled with a user-friendly control panel, allowing users to dynamically adjust ecological parameters and save scenarios without requiring programming expertise. The primary objectives are to observe ecosystem stability, extinction events, and "boom and bust" population cycles under varied conditions.

Keywords: agent-based modeling, predator-prey dynamics, ecosystem simulation, population dynamics, Python, visual environment, configurable ecological research

1 Project Overview: Simulating the Circle of Life

1.1 The Background

In nature, the relationship between predators and their prey is a delicate balancing act. When prey populations (such as rabbits) grow, predators (such as foxes) have more food and multiply. When predators eat too much, food runs out, and their numbers drop. For a long time, scientists used mathematical equations to study these cycles. While those equations were a great starting point, they had a major flaw: they treated animals as perfectly mixed numbers on a page. They did not account for real factors such as physical space, the effort required to hunt, or simple random chance.

1.2 The Modern Approach: Virtual Ecosystems

To get a more realistic picture, modern researchers use "agent-based models". Instead of using abstract math, this approach works a bit like a video game, creating a virtual world filled with independent, autonomous agents. Due to its flexibility, agent-based modeling has become

a widely used tool in many fields. In evolutionary modeling, it has been applied to the mechanisms of sexual selection and pair formation [2], including individual-based quantitative-genetic simulations [1].

In ecological contexts, agent-based models enable the study of predator-prey coevolution in physically simulated environments. Previous work has demonstrated how predator and prey populations co-evolve through adaptation [4, 6]. Furthermore, Van der Laan and Hogeweg [7] showed how ecological and evolutionary dynamics can interact across different timescales.

More recently, these models have been extended by incorporating artificial intelligence techniques. Reinforcement learning, for example, has been used to study the co-evolution of predator-prey ecosystems in which agents actively adapt their strategies [5]. Similarly, predator-induced survival pressure alone has been shown to lead to the emergence of complex collective behaviors such as swarming [3].

1.3 What This Project Does

Following this modern and open approach, this project focuses on building an agent-based simulation of a simple food chain. It includes three levels:

- **Grass** that slowly grows back over time.
- **Prey** that wander around eating the grass.
- **Predators** that hunt prey.

Every action an animal takes costs energy, and the only way they can survive is by finding food.

The main goals of this project are:

- To build a working, easy-to-use computer simulation of this predator-prey relationship.
- To test what happens when we tweak the rules of nature. For example, what happens if grass grows slower? What if predators need more food to survive?
- To see if our virtual world ends in a stable balance, constant population rollercoasters, or total extinction.
- To explore how individual agent rules lead to complex, ecosystem-wide consequences, drawing inspiration from modern computational studies.

1.4 What Was Built

To make this happen, the project includes the following:

- **A fully functioning virtual environment** programmed in Python that acts as a visual grid where animals move and interact.
- **An explicit ecosystem** where grass grows continuously, prey consume grass, and predators consume prey.
- **A gene-driven energy system** where traits like metabolism and reproduction evolve dynamically from user-defined starting parameters.

- **A user-friendly control panel** that lets users easily change the simulation's settings and save their favorite scenarios without needing to code.
- **A genetic evolution system** where random mutations at birth drive ongoing predator-prey coevolution purely through natural survival.
- **A live analytics dashboard** featuring real-time population charts, death statistics by cause, and point-and-click genotype inspection.

2 Problem formulation and proposed solution

This section formalises the research problem and details the proposed simulation, building upon the motivation and literature review from Section 1. The following subsections describe the system architecture (2.2), the formal equations governing agent interactions (2.3), and the core step dynamics alongside the genetic mutation scheme (2.4).

2.1 Research problem formulation

Classical differential equation models, such as the Lotka–Volterra system, assume populations are continuous and perfectly mixed. While useful for basic analysis, they fail to capture spatial structure, individual variation, and the randomness of local interactions [7].

This project addresses these limitations. The core research question is: **How do population-level dynamics emerge from the purely local interactions of individual prey and predators, each possessing an evolvable genotype?**

Specifically, the study investigates:

- Under which parameter conditions does the ecosystem stabilise, oscillate, or collapse?
- What specific genetic traits does natural selection favour for each species?
- Can different survival strategies (e.g., living longer with a low metabolism vs. living shorter but reproducing rapidly) emerge spontaneously, without being explicitly programmed [6]?

Proposed solution. To answer these questions, an agent-based simulation implemented in Python is proposed. The environment is a 2D grid with wrap-around edges (a torus), populated by three interacting entities: grass cells, prey, and predators. Unlike classical models, every animal carries a unique genotype. When animals reproduce, their genes mutate via multiplicative Gaussian noise, following the quantitative-genetic approach of [1]. This mechanism drives continuous predator–prey coevolution based solely on local survival and reproduction pressures [6, 4, 3].

2.2 System architecture

The system is built around three distinct logical layers, following the single-responsibility principle:

- **Configuration layer.** A `pygame_gui`-based setup interface (`config_screen.py`) where the user adjusts initial parameters (population sizes, mutation bounds, gene values) and selects saved scenarios. Configurations are saved as JSON files.

- **Simulation engine.** The core logic, encapsulated within the `Simulation` class (`simulation.py`). At each time step, it updates grass growth and handles all animal interactions: movement, feeding, reproduction, ageing, and death.
- **Renderer and analytics.** Managed by the `SimRenderer` class, this layer handles the visual output. It draws the 2D spatial grid and updates the interactive side-panel with real-time charts, death statistics, and genetic data. Users can also click on any agent to inspect its exact genes.

External data is kept separate from the main codebase: parameter definitions are in `configs/parameters/default.json`, UI layouts in `configs/layout.json`, and gene descriptions in `configs/genes_info.json`. This clean separation makes it easy to maintain the code, run reproducible experiments, and test different ecological scenarios.

2.3 Simulation mechanics

The simulation operates on a toroidal (wrap-around) 2D grid. Each animal has a state consisting of its position, energy, age, and a dictionary of genes. The rules described below directly reflect the logic implemented in the Python engine.

Movement. An animal chooses a random direction from the 8 neighboring cells. The number of cells it actually moves (n) depends on its `move_distance` gene d :

$$n = \max(1, \lfloor d \rfloor + X) \quad (1)$$

where $X = 1$ with probability $d - \lfloor d \rfloor$, and 0 otherwise. This allows fractional gene values to translate into discrete grid steps (e.g., a distance of 1.7 means 1 step plus a 70% chance for a second step). Movement drains energy based on the distance:

$$E_{\text{new}} = E_{\text{old}} - (\text{move_cost} \cdot n) \quad (2)$$

Encounters and feeding. If a predator and a prey occupy the same cell, the predator catches the prey with the following probability:

$$P(\text{catch}) = \text{hunt_success} \cdot (1 - \text{escape_chance}) \quad (3)$$

If successful, the prey dies immediately, and the predator gains energy (capped at its maximum):

$$E_{\text{new}} = \min(\text{max_energy}, E_{\text{old}} + \text{eat_energy}) \quad (4)$$

Prey eat from their current cell whenever the local grass level exceeds 0.2; a successful meal reduces the cell's grass by 0.4 and grants the prey `eat_energy` energy (capped at `max_energy`).

Ageing and death. Once an animal's age exceeds its `max_age` gene, it faces a growing probability of natural death per step:

$$P(\text{death}) = \min \left(1, 0.05 \cdot \left(\frac{\text{age}}{\text{max_age}} - 1 \right) \right) \quad (5)$$

Reproduction. If an animal's energy is at least `reproduce_energy`, it attempts to reproduce with a probability equal to its `reproduce_chance` gene. If successful, the parent transfers a fraction of its energy to the newborn child:

$$E_{\text{child}} = \lfloor E_{\text{parent}} \cdot \text{offspring_investment} \rfloor \quad (6)$$

The parent's energy is reduced by exactly E_{child} to ensure strict energy conservation within the ecosystem.

Genetic mutation. A newborn inherits its parent's genes, but a random number of them (between `mutation_min` and `mutation_max`) undergo mutation. Each selected gene is multiplied by a Gaussian random variable:

$$g_{\text{new}} = \max(0, g_{\text{old}} \cdot (1 + N(0, \sigma))) \quad (7)$$

where $N(0, \sigma)$ is a normal distribution with mean 0 and standard deviation σ (`mutation_strength`). Genes representing probabilities (like `escape_chance` or `reproduce_chance`) are additionally capped at 1.0.

Metabolism. Regardless of the actions taken, every animal pays a baseline metabolic cost at each time step to sustain its life functions. This is subtracted continuously:

$$E_{\text{new}} = E_{\text{old}} - \text{metabolism} \quad (8)$$

If an animal's energy drops to zero or below as a result of movement or metabolism, it dies of starvation.

2.4 Algorithms

The genetic mutation mechanics described in Equation (7) are implemented directly in Python, as shown in Listing 1. The function selects a random subset of genes and applies the multiplicative Gaussian noise, ensuring that probability-based traits remain correctly bounded within the $[0, 1]$ range.

```

1 def mutate_genes(parent_genes, cfg):
2     child = dict(parent_genes)
3     n_min = int(cfg["mutation_min"])
4     n_max = int(cfg["mutation_max"])
5     strength = cfg["mutation_strength"]
6     n = random.randint(min(n_min, n_max), max(n_min, n_max))
7     if n > 0:
8         keys = random.sample(list(child.keys()), min(n, len(child)))
9         for k in keys:
10             child[k] = max(0.0, child[k] * (1 + random.gauss(0, strength)))
11     # probability-valued genes clamped to [0, 1]
12     for k in ("reproduce_chance", "escape_chance",
13             "hunt_success", "offspring_investment"):
14         if k in child:
15             child[k] = min(1.0, child[k])
16     return child

```

Listing 1: Multiplicative Gaussian mutation applied to an offspring's genes. Source: own work.

The overall progression of the ecosystem is driven by the engine's `step()` method. To provide a clear overview of the system's dynamics, Algorithm 1 outlines the high-level logical flow of a single simulation tick, focusing on the core ecological rules rather than technical details like spatial indexing.

Algorithm 1: Description of a single simulation step.

Input: simulation state (grass grid, animal population), configuration

Output: updated simulation state

```

1 Regrow grass uniformly across all grid cells;
2 Build spatial index of currently alive animals;
3 foreach alive animal a in the population do
4   Move a by random direction and distance  $n_a$  (Eq. 1–2);
5   Increase age and evaluate natural senescence (Eq. 5);
6   if a survives then
7     if a is prey then
8       Eat grass at current cell if available (Eq. 4);
9       Deduct baseline metabolic cost  $\mu_a$ ;
10      if predator p is present and  $U(0, 1) < P(\text{catch})$  (Eq. 3) then
11         $p$  gains energy; mark a as dead;
12      end
13    end
14    else
15      Deduct baseline metabolic cost  $\mu_a$ ;
16      if prey q is present and  $U(0, 1) < P(\text{catch})$  then
17         $a$  gains energy; mark q as dead;
18      end
19    end
20    if  $\text{energy} \geq \theta_a$  and  $U(0, 1) < r_a$  then
21      Generate offspring with mutated genotype  $\mathbf{g}_{\text{child}}$  (Eq. 7);
22    end
23  end
24 end
25 Remove dead or starved animals from the population;
26 Integrate newborns into the population;
27 Log aggregated population statistics;

```

2.5 Libraries and tooling

The implementation relies on a small set of well-established Python libraries:

- **pygame 2.6** - the visual rendering backend and the input/event loop. Visual elements like cells, animals, and the chart are drawn using standard `pygame.draw` functions.
- **pygame_gui** - handles UI widgets for the configuration screen and the analytics panel (labels, buttons, text fields, dropdowns). It also automatically manages styling, such as highlighting the active tab.
- **Python standard library** - `random` for random events and mutations, `collections.deque` for history buffers, `json` for loading/saving configurations, and `csv` as a planned data export format.

No external numerical or machine-learning libraries are used. All simulation logic is written in plain Python using standard lists and dictionaries. This keeps the model completely transparent and easy to modify, at the cost of larger simulations being CPU-bound.

3 Experimental research results

This section details the simulation’s behavior under controlled conditions and addresses the research questions posed in Section 2.1. The model was evaluated by directly observing the simulation via an interactive application - its configuration screen and live view are shown in Figures 1 and 2.

3.0.1 Experimental setup.

The simulation operates using the project’s default configuration parameters. The environment consists of a 60×40 toroidal grid with a grass regrowth rate of 0.03. The initial population is set to 80 prey and 20 predators. The model also utilizes the default gene values detailed on the application’s *Genes* tab. Each simulation run lasts for 1500 steps, which provides sufficient time to observe several full population cycles.

3.1 Figures and tables

3.1.1 Figures

Figure 1 shows the configuration screen for managing ecological parameters and scenarios. Figure 2 illustrates the simulation during two characteristic phases: a prey boom with heavily grazed grass Figure 2 (a), followed by a predator boom that causes a prey collapse and allows grass recovery Figure 2 (b). The accompanying side panel displays real-time statistics, including population charts, death causes, and average genotypes.

The dynamics of the ecosystem are captured in three key plots. Figure 3 highlights the boom-and-bust nature of the system through a representative population time series. Figure 4 tracks the average values of genes over time, demonstrating the action of evolutionary selection on agents. Finally, Figure 5 compares population dynamics across three pre-configured scenarios to illustrate how different initial settings alter the simulation’s behavior.

3.1.2 Tables

The simulation results are further detailed in four tables. Table 1 explores the availability of resources by showing the impact of varying the grass regrowth rate. Table 2 tracks changes in the population’s average genotype from the beginning to the end of the simulation. Table 3 provides a breakdown of animal mortality by cause. Finally, Table 4 summarizes the outcomes of the pre-configured scenarios.

3.2 Analysis and discussion

3.2.1 Population dynamics and stability

The default simulation does not settle into a static balance. Instead, it produces continuous boom-and-bust cycles, as shown in Figure 3. A prey boom depletes the grass and leads to a delayed increase in predators. The predators then reduce the prey population, allowing the

PREDATOR-PREY COEVOLUTION SIMULATION

Configuration file: Save as:

Configuration **Genes**

General / Simulation Settings

Grid width	60	Grid height	40
Grass regrowth rate	0.030	Grass eat threshold	0.200
Grass eat amount	0.400	Simulation speed	10
History length	300		

Predator Settings

Initial predators	20	Predator max energy	150
Predator eat prey energy	60	Predator move cost	2
Predator reproduce chance	0.020	Predator reproduce energy	100
Predator metabolism	3	Predator hunt success	0.900
Predator move distance	1.000	Predator max age	300
Predator offspring investment	0.400		

Prey Settings

Initial prey	80	Prey max energy	100
Prey eat grass energy	30	Prey move cost	1
Prey reproduce chance	0.040	Prey reproduce energy	60
Prey metabolism	1	Prey escape chance	0.100
Prey move distance	1.000	Prey max age	200
Prey offspring investment	0.400		

Mutation Settings

Mutation min	0	Mutation max	3
Mutation strength	0.100		

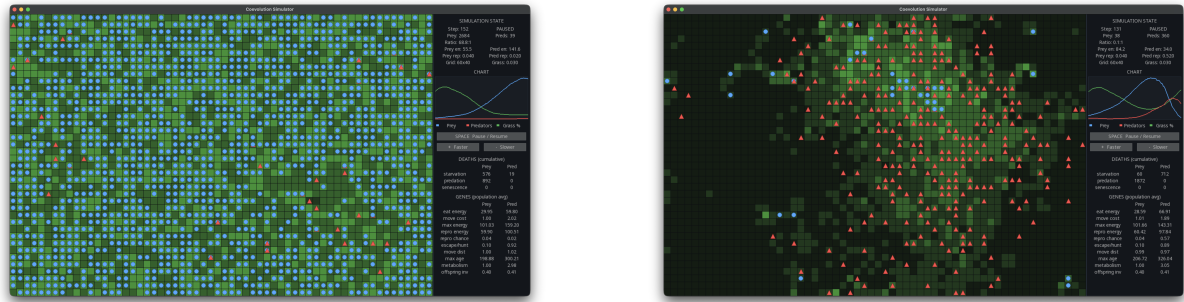
Set parameters and click START

Figure 1: Configuration screen of the simulator. The user edits ecological parameters grouped into general, predator, prey and mutation sections, and can save or load named scenarios before starting a run

grass to recover and starting the next cycle. This delay between prey and predator peaks resembles classic Lotka–Volterra cycles, but here it emerges purely from local interactions rather than mathematical equations. Despite these large population fluctuations, the ecosystem remains highly stable. As seen in Table 1, both species survived in all grass regrowth rates tested. Therefore, the ecosystem continues to cycle instead of collapsing, which answers the first research question.

3.2.2 Effect of resource availability

Table 1 shows that the rate of grass regrowth controls how many animals the environment can support. When the rate increases from 0.01 to 0.08, the average prey population grows more than six times (from 361 to 2397), and the grass cover roughly doubles. However, the predator population remains roughly the same (between 117 and 156). This difference occurs because the number of prey depend directly on food (grass). Predators, on the other hand, are limited by their high energy needs, not by a lack of prey. Therefore, having more prey leads to bigger population swings rather than many more predators. This matches the causes of death discussed next.



(a) Prey boom: many prey (blue circles) fill the grid and eat most of the grass.

(b) Prey crash: many predators (red triangles) eat the prey, letting the grass grow back.

Figure 2: The simulation in two phases of one population cycle. The map is on the left and live stats are on the right

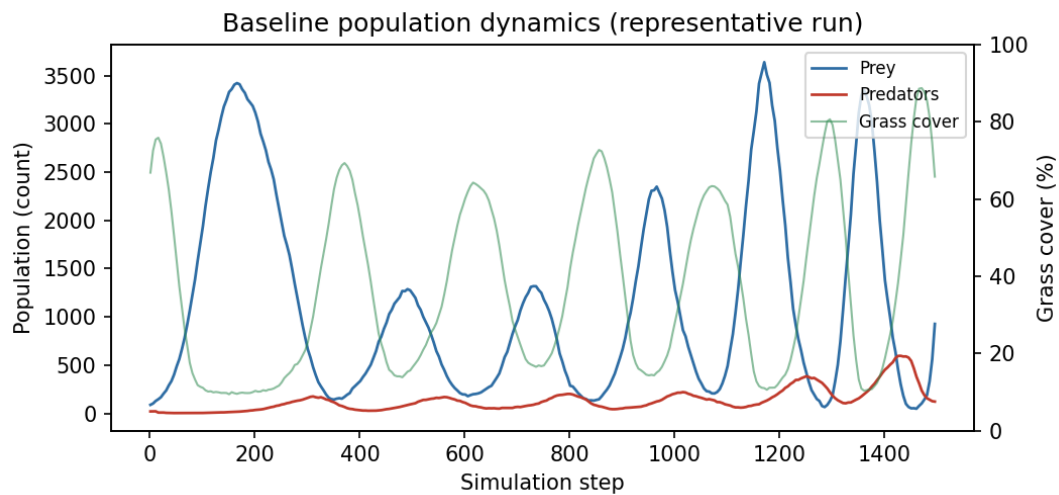


Figure 3: Population changes over time for a standard run (60×40 grid, regrowth rate 0.03, 1500 steps). Prey and predator numbers are on the left axis; grass cover is on the right axis. Predator peaks happen just after prey peaks, showing a typical predator-prey cycle

3.2.3 Mortality structure

Table 3 shows a clear difference between the two species. Most prey are eaten by predators (82.6%) or die of starvation (17.2%). Very few die of old age (0.2%) because predators usually catch them first. Predators show the exact opposite pattern, dying almost entirely from starvation (98.5%). In short, the number of prey is controlled by hunters, while the number of predators is limited by the finding of enough food. This creates a realistic food chain.

3.2.4 Emergent selection and coevolution

Because offspring inherit the mutated genes of their parents, the population naturally evolves over time without any programmed goals [1, 6]. Table 2 and Figure 4 show that general survival traits (such as breeding speed and energy use) change much more than specific hunting or escaping skills. For example, prey reproduce_chance jumping from 0.040 to 0.151, under heavy predator threat, the prey that survive are the ones that breed as fast as possible. This is known as an r-selected strategy: producing many offspring quickly rather than a few strong

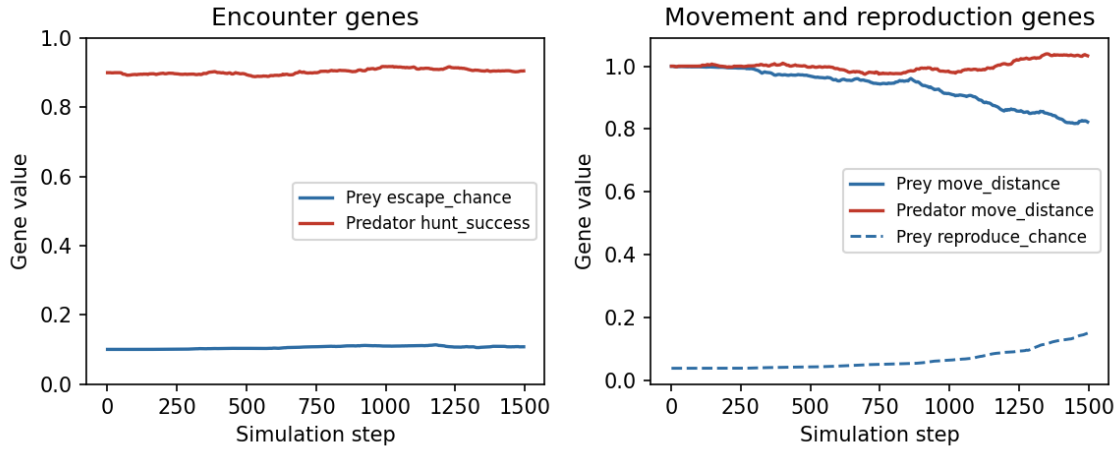


Figure 4: Changes in average gene values over time. Left - encounter genes (prey `escape_chance` and predator `hunt_success`), which barely change. Right - prey and predator `move_distance` and prey `reproduce_chance` (dashed line), which change a lot

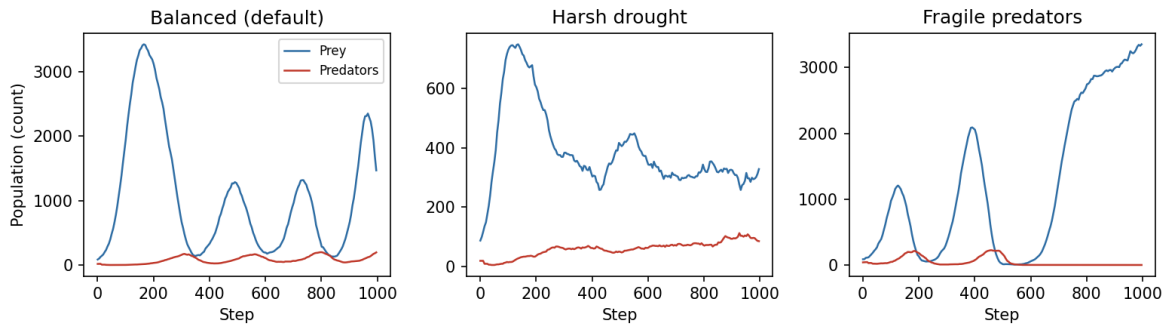


Figure 5: Population changes over time for three pre-made scenarios (1000 steps). The default scenario goes up and down with both species alive. The drought scenario has a much smaller and stable ecosystem. In the fragile predators scenario, predators die out around step 500, and then the prey population grows without limit

ones.

Both species also lower their metabolism (prey by 11%, predators by 16%). This saves energy and helps them survive longer between meals. Since most predators die from starvation, this is their most important adaptation. The prey also evolve a shorter `move_distance`, saving energy and reducing the risk of falling into a predator.

In contrast, the expected "arms race" between hunters and prey is very weak. Prey `escape_chance` increases only slightly (+8%) and predator `hunt_success` barely changes because it starts near its maximum limit of 1.0. In short, saving energy and breeding faster are much better survival strategies here than improving chase skills. This answers our third research question, proving that complex survival strategies can emerge naturally from simple local rules [3].

3.2.5 Bundled scenarios and an extinction event

The project includes five pre-made scenarios to easily test different ecosystem behaviors. Their results are shown in Table 4 and Figure 5. The *abundant meadow* (plentiful grass) supports a large and lively ecosystem, while the *harsh drought* (scarce grass) creates a much smaller and more stable one. In the *fast evolution* scenario, a higher mutation rate causes the prey to

Table 1: How the grass regrowth rate affects the ecosystem (1500 steps). The table shows the average number of prey, predators, and grass cover. Both species survived in all cases

Regrowth rate	Both species survived	Mean prey	Mean predators	Mean grass cover
0.01	Yes	361	117	20.3 %
0.02	Yes	551	146	33.2 %
0.03	Yes	922	142	38.0 %
0.05	Yes	1209	156	45.9 %
0.08	Yes	2397	122	44.2 %

Table 2: Average gene values at the start and after 1500 steps. The last column shows the percentage change

Species	Gene	Start	End	Change
Prey	reproduce_chance	0.040	0.151	+279 %
	move_distance	1.000	0.823	−18 %
	metabolism	1.000	0.890	−11 %
	offspring_investment	0.400	0.362	−10 %
	escape_chance	0.100	0.108	+8 %
	max_age	200	196	−2 %
Predator	metabolism	3.000	2.534	−16 %
	reproduce_chance	0.020	0.022	+12 %
	max_age	300	277	−8 %
	offspring_investment	0.400	0.426	+7 %
	move_distance	1.000	1.033	+3 %
	hunt_success	0.900	0.905	+1 %

breed so fast that their population explodes, showing how rapid mutations can destabilize the system.

The most interesting case is *those of fragile predators*. Here, predators are highly deadly and prey are almost defenseless. Instead of creating a predator paradise, predators hunt, destroy their own food supply and starve to death (Table 4). Once predators die out, the prey population grows without limits, as seen in the runaway blue curve in Figure 5. This is exactly the kind of extinction event the project aimed to study, proving that being too good at hunting is ultimately self-defeating.

4 Summary and conclusions

This project aimed to build an accessible predator-prey simulation to study how population patterns emerge from local interactions and evolving genes. The final system meets this goal. It features a configurable ecosystem of grass, prey, and predators, where every action costs energy. It includes genetic inheritance with mutations and an interactive interface with live charts and statistics, all usable without writing any code.

The complete source code, configuration files, and the interactive environment are publicly available on GitHub at https://github.com/przemuuu/System_Modeling_and_Simulation_Project.

Table 3: Causes of death for prey and predators. Predators are never hunted, so their predation count is zero

Cause of death	Prey	Predators
Predation	82.6 %	0.0 %
Starvation	17.2 %	98.5 %
Senescence	0.2 %	1.5 %
Total deaths	433 052	15 497

Table 4: Results of the five pre-made scenarios after 1000 steps. The table shows the average number of prey, predators, and grass cover, as well as whether both species survived.

Scenario	Both species survived	Mean prey	Mean predators	Mean grass cover
Balanced (default)	Yes	820	100	37.0 %
Abundant meadow	Yes	1602	110	44.3 %
Harsh drought	Yes	369	76	18.4 %
Fragile predators	No	2708	19	27.2 %
Fast evolution	Yes	5010	749	25.8 %

The experiments support three main conclusions. First, the model naturally reproduces the classic boom-and-bust cycles of predator-prey systems (Figure 3), and the default ecosystem is highly stable rather than collapsing. Second, resource availability controls the ecosystem’s size: faster-growing grass multiplies the prey population, while the predator population stays mostly flat due to its high energy costs. Third, population control works both ways: prey are limited by hunters, while predators are limited by starvation (Table 3). Under these pressures, animals naturally evolve clear survival strategies, such as faster prey breeding and lower metabolism for both species (Table 2), without any programmed goals.

Several limitations point to future work. Testing more parameters, like mutation strength, predator efficiency, and initial populations over longer periods would make the evolutionary conclusions even stronger. Technically, the model is written in plain Python and slows down when populations reach several thousands. Optimizing the code would allow for larger maps and longer simulations. Finally, the animals genes could be expanded to include new skills, like sensing food or actively running away from threats. Adding a data export feature and a map showing where specific genes are concentrated would also help connect the evolutionary results with the visual patterns seen in Figure 2.

References

- [1] D. van Dijk, P. M.A. Sloot, J. C. Tay, and M. C. Shut. “Individual-based simulation of sexual selection: A quantitative genetic approach”. In: *Procedia Computer Science* (2010). URL: <https://www.sciencedirect.com/science/article/pii/S1877050910002255>.
- [2] R. Dreżewski. “The agent-based model and simulation of sexual selection and pair formation mechanisms”. In: *Entropy* (2018). URL: <https://www.mdpi.com/1099-4300/20/5/342>.

-
- [3] J. Li, L. Li, and S. Zhao. “Predator-prey survival pressure is sufficient to evolve swarming behaviors”. In: *New Journal of Physics* (2023). URL: https://shiyuzhao.westlake.edu.cn/Li_2023_New_J._Phys._25_092001.pdf.
 - [4] N. Ouannes, N. Djedi, Y. Duthen, and H. Luga. “Predator-Prey Coevolution in a Physically Simulated Environment”. In: *Conference Paper* (2015). URL: https://www.researchgate.net/publication/281843881_Predator-Prey_Coevolution_in_a_Physically_Simulated_Environment.
 - [5] J. Park, J. Lee, K. Taehwan, I. Ahn, and J. Park. “Co-Evolution of Predator-Prey Ecosystems by Reinforcement Learning Agents”. In: *Entropy* (2021). URL: <https://www.mdpi.com/1099-4300/23/4/461>.
 - [6] I. Takashi, M. L. Pilat, R. Suzuki, and T. Arita. “ALife approach for body-behavior predator-prey coevolution: body first or behavior first?” In: *The Eighteenth International Symposium on Artificial Life and Robotics* (2013). URL: <https://alife-robotics.co.jp/members2013/icarob/data/papers/GS17/GS17-4.pdf>.
 - [7] J. D. Van der Laan and P. Hogeweg. “Predator-prey coevolution: interactions across different timescales”. In: *Proceedings of the Royal Society of London* (1995). URL: <https://www-binf.bio.uu.nl/pdf/Vanderlaan.prs1b95-259.pdf>.